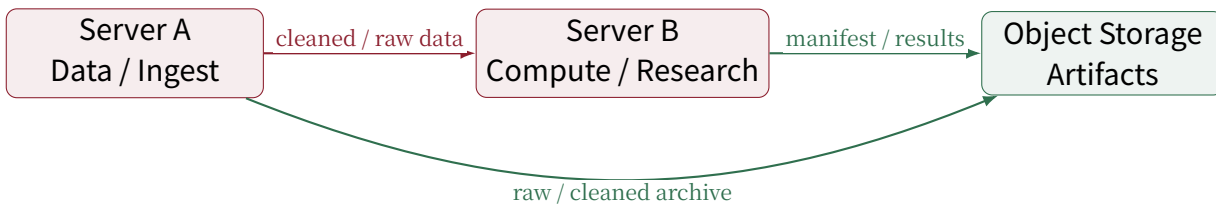


量化投研平台服务器治理 与模块协作 API 规范

Server A / Server B 部署、统一环境、Job API、Event 与结果权限管理

文档定位：本规范用于指导量化投研平台从单用户服务器形态迁移为团队级云端工程平台。重点覆盖服务器治理、模块接口治理、任务执行治理、事件通知治理、结果权限治理与后续维护人员协作规范。



适用对象	项目负责人、基建组、模块负责人、后续维护人员
服务器配置	腾讯云 Server A / Server B, 均为 8C32G
核心机制	API / Job API / Event / Manifest / systemd / RBAC
版本	v3.0 - Platform Governance
日期	2026 年 5 月 2 日

目录

1	文档目标与治理原则	3
1.1	文档目标	3
1.2	治理原则	3
2	服务器拓扑与固定配置	3
2.1	Server A / Server B 固定配置	3
2.2	网络与存储边界	3
2.3	部署拓扑图	4
3	项目模块分层与部署映射	4
3.1	模块分层	4
3.2	Server A 部署映射	4
3.3	Server B 部署映射	5
3.4	代码、配置、数据、日志分离	5
4	统一 Environment 与配置管理	6
4.1	统一 Environment 策略	6
4.2	推荐规范	6
4.3	配置文件模板	6
5	权限、用户组与服务账号	6
5.1	人员账号原则	6
5.2	用户组设计	7
5.3	服务账号设计	7
5.4	目录权限矩阵	8
6	模块 API 文档规范	8
6.1	为什么必须先写接口文档	8
6.2	INTERFACE.md 模板	8
6.3	模块接口文档清单	9
7	API / Job API / Event 分工	10
7.1	三者的边界	10
7.2	Query API	10
7.3	Job API	10
7.4	Event	10
7.5	典型调用链路	11
8	Job 执行、Manifest 与结果权限分层	11
8.1	Job Request 标准格式	11
8.2	Job 状态机	12
8.3	参数白名单与资源限额	12
8.4	Manifest 标准	12
8.5	结果权限分层	13
8.6	服务化协作方式	13

9	迁移、上线与回滚流程	13
9.1	迁移目标	13
9.2	迁移流程	13
9.3	迁移验收 Smoke Test	13
10	监控、告警与后续维护人员要求	14
10.1	systemd 服务托管	14
10.2	监控与告警清单	14
10.3	后续维护人员分级	15
10.4	后续维护人员必备技术栈	15
	附录：推荐落地顺序	15

1 文档目标与治理原则

1.1 文档目标

本规范的目标是建立一套可执行的服务器治理与模块协作标准，使量化投研平台在多人协作、双服务器部署和多模块联调场景下保持稳定、可审计、可回滚和可扩展。

核心治理目标

代码可追踪，数据可回滚，任务可审计，结果可分层，权限可回收。

本规范重点解决以下问题：

- Server A / Server B 的职责边界如何划分；
- 不同模块如何在不暴露内部目录和密钥的前提下协作；
- 同步查询、异步任务、事件通知三类接口如何分工；
- 任务执行结果如何通过 Manifest 暴露给下游；
- 后续维护人员如何安全地查看、运行、排查和交接系统。

1.2 治理原则

原则	说明
人员与服务分离	人员账号用于登录、审查和运维；服务账号用于运行生产服务。生产服务不得使用个人账号运行。
代码与数据分离	代码通过 Git 管理；运行数据、回测结果、模型文件和日志进入 workspace_data、数据盘或对象存储。
配置与代码分离	.env.example 进入 Git；真实 prod.env 放在服务器受控目录，不进入 Git。
同步与异步分离	轻量查询使用 API；耗时任务使用 Job API；产物就绪使用 Event 通知。
结果默认受限	Job 原始结果默认 private；下游只通过 Manifest 和授权投影结果读取。
操作必须留痕	Job、服务重启、配置变更、数据迁移和权限变更必须保留审计记录。

2 服务器拓扑与固定配置

2.1 Server A / Server B 固定配置

平台采用两台腾讯云服务器作为常驻运行资源。两台服务器配置统一为 8C32G, 不再设置增强规格与 MVP 规格的区分。

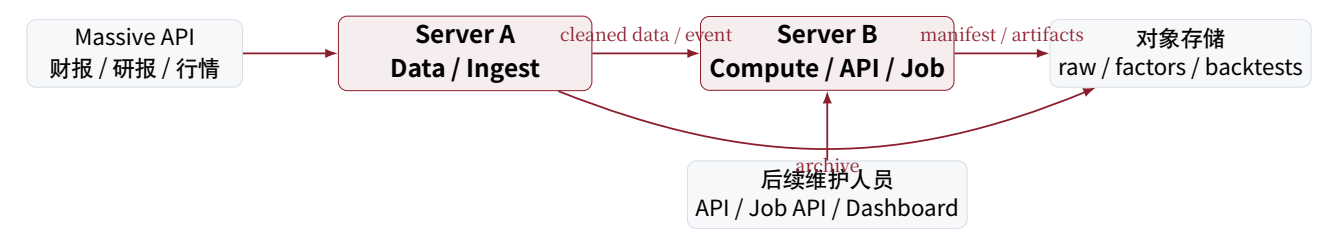
服务器	固定配置	定位	主要职责
Server A	8 vCPU / 32GB RAM	Data / Ingest Server	数据接入、原始数据下载、校验、清洗、Parquet 落盘、热数据缓存、对象存储归档。
Server B	8 vCPU / 32GB RAM	Compute / Research / Service Server	因子计算、因子评估、因子准入、策略信号、回测、API、Job API、Event 处理和服务调度。

2.2 网络与存储边界

- 公网带宽按每台服务器 30-50 Mbps 规划，用于 API 访问、数据下载、对象存储同步和运维访问。
- Server A 与 Server B 应部署在同一 VPC 内，优先使用内网通信，避免模块之间通过公网互相访问。
- 每台服务器配置 100GB 系统盘，数据盘建议 500GB-1TB 起步，用于热数据、缓存、运行产物和临时计算。

- 长期历史数据、回测结果、模型文件和审计归档应进入对象存储，不应长期堆积在系统盘或 Git 工作区。

2.3 部署拓扑图



3 项目模块分层与部署映射

3.1 模块分层

项目后端采用研究链路式分层，主要包括数据层、因子层、策略层、回测层、Demo 与统一运行产物目录。

模块	职责	部署与协作含义
raw_data_layer	原始数据抓取、校验、清洗和样例数据管理。	主要部署在 Server A；输出 raw / cleaned 数据，并通过 Event 通知下游。
factor_layer	因子计算、Agent 自动化 YAML 生成、因子评估、因子准入和特化因子项目。	主要部署在 Server B；应暴露 factor-computation、factor-evaluation、factor-admission、agent-report-parse 等 Job。
strategy_layer	将因子值、行情和辅助数据转换成单资产目标仓位或组合 holdings。	部署在 Server B；消费 factor lake 和 cleaned 数据，输出 signals、target_position、holdings。
backtest_layer	将目标仓位或 holdings 转换为 returns、metrics、summary 和审计元数据。	部署在 Server B；应作为标准 backtest Job 的执行模块。
demo	端到端 smoke demo 与 notebook 展示。	用于迁移验收、回归测试和新成员理解主链路，不作为生产常驻服务。
workspace_data	统一运行时产物根目录。	服务器上应指向数据盘，统一承载 demos、factors、strategy、backtests、cache 等产物。

3.2 Server A 部署映射

```
/srv/quant/repos/quantsociety_backend/  
  raw_data_layer/  
  workspace_data -> /srv/quant/data/workspace_data  
  
/srv/quant/configs/  
  data/prod.env  
  
/srv/quant/envs/  
  platform/          # 短期统一环境  
  
/srv/quant/data/  
  raw/
```

```
cleaned/  
hot/  
workspace_data/  
  
/srv/quant/logs/  
data/
```

Server A 的重点是数据入口和数据质量，不承担大规模研究计算。Server A 上的生产任务应尽量保持少而稳定，避免后续维护人员在数据入口服务器上进行大量 notebook 实验或全市场回测。

3.3 Server B 部署映射

```
/srv/quant/repos/quant society_backend/  
  factor_layer/  
  strategy_layer/  
  backtest_layer/  
  demo/  
  workspace_data -> /srv/quant/data/workspace_data  
  
/srv/quant/configs/  
  factor/prod.env  
  strategy/prod.env  
  backtest/prod.env  
  job_api/prod.env  
  
/srv/quant/envs/  
  platform/           # 短期统一环境  
  
/srv/quant/results/  
  private/  
  internal/  
  public/
```

Server B 是计算、研究和服务调度中心。API、Job API、Event handler、因子评估、策略生成和回测服务均应优先部署在 Server B。

3.4 代码、配置、数据、日志分离

类型	推荐路径	说明
代码	/srv/quant/repos/ quant society_backend	Git 管理；不写入真实密钥和大规模运行产物。
配置	/srv/quant/configs/ <module>/prod.env	真实生产配置；仅管理员和对应服务账号可读。
运行产物	/srv/quant/data/ workspace_data	对应项目统一产物目录，建议由 Git 内软链接指向。
结果分层	/srv/quant/results/ private internal public	Job 输出和对外投影的权限分层目录。
日志	/srv/quant/logs/ <service>	服务日志、Job 日志和审计日志。
备份	/srv/quant/backups	临时迁移备份；长期备份应进入对象存储和云快照。

4 统一 Environment 与配置管理

4.1 统一 Environment 策略

当前阶段优先采用统一平台环境，减少多人协作、迁移和联调成本。长期在服务边界稳定后，可逐步拆分为 data、factor、strategy、backtest、job_api 等服务级环境。

阶段	Environment 策略
短期迁移阶段	使用统一 Python Runtime 和统一平台环境，例如 /srv/quant/envs/platform/，确保所有模块能按同一依赖口径跑通。
中期稳定阶段	保持统一 Python 主版本和统一配置规范；对资源边界清晰的模块逐步拆分服务级环境。
长期生产阶段	形成平台统一基础镜像 + 模块独立依赖锁定，避免单个模块升级影响全平台。

4.2 推荐规范

- Python 主版本统一，例如 Python 3.11。
- 根目录保留统一依赖入口；开发测试依赖与运行依赖应清晰区分。
- 每个模块必须提供 .env.example，声明需要的环境变量。
- 真实生产配置统一放在 /srv/quant/configs/<module>/prod.env。
- systemd 服务统一通过 EnvironmentFile= 加载生产环境变量。
- 禁止将真实 prod.env、API Key、云密钥、数据库密码写入 Git。

4.3 配置文件模板

```
# .env.example - 进入 Git，只放占位符
APP_ENV=prod
LOG_LEVEL=INFO
WORKSPACE_DATA_ROOT=/srv/quant/data/workspace_data
RESULT_ROOT=/srv/quant/results
OBJECT_STORAGE_BUCKET=replace_me
OBJECT_STORAGE_REGION=ap-hongkong
JOB_API_BASE_URL=http://127.0.0.1:8000
```

```
# /srv/quant/configs/factor/prod.env - 不进入 Git
APP_ENV=prod
LOG_LEVEL=INFO
WORKSPACE_DATA_ROOT=/srv/quant/data/workspace_data
RESULT_ROOT=/srv/quant/results
FACTOR_ENGINE_BACKEND=pandas
OBJECT_STORAGE_BUCKET=quant-prod-artifacts
OBJECT_STORAGE_SECRET_ID=***
OBJECT_STORAGE_SECRET_KEY=***
```

5 权限、用户组与服务账号

5.1 人员账号原则

每位后续维护人员应使用独立 Linux 用户登录服务器，禁止多人共享同一用户。共享用户会导致操作不可审计、权限不可回收、责任不可追踪。

原则	说明
一人一账号	登录、审计和权限回收均以个人账号为单位。
服务账号独立	生产服务使用 service account 运行，不使用个人账号。
最小权限	默认只给完成任务所需的最小目录、命令和结果权限。
禁用 root 直登	root 不直接 SSH 登录，管理操作通过 sudo 审计。
密钥可回收	成员离开项目时移除 SSH 公钥、GitHub 权限和服务器组权限。

5.2 用户组设计

用户组	职责	权限边界
quant-admin	服务器管理员与核心负责人。	sudo、配置管理、权限管理、迁移和回滚。
quant-dev	模块开发与代码维护。	代码目录读写；无生产密钥默认权限。
quant-research	研究任务、因子和回测使用者。	读取 internal 结果，提交 Job，不直接改生产配置。
quant-ops	后续运维维护人员。	查看日志、检查服务、重启授权服务。
quant-readonly	只读观察者。	只能读取 public / 部分 internal 摘要。

5.3 服务账号设计

服务账号	运行对象	权限说明
svc-data	数据下载、清洗、校验和归档任务。	可写 raw / cleaned / cache；不可读 Agent 或模型密钥。
svc-factor	因子计算、评估、准入任务。	可读 cleaned / factor lake；可写 factor results。
svc-strategy	策略信号、组合信号和 holdings 生成。	可读 admitted factors；可写 strategy results。
svc-backtest	回测任务。	可读策略输出和行情；可写 backtest results。
svc-jobapi	Job API 与调度器。	可提交任务、读写 run metadata、调用 worker。
svc-dashboard	Dashboard / API 展示层。	只读 public / 授权 internal 摘要，不读 private。

5.4 目录权限矩阵

目录	Owner	Group	权限建议
/srv/quant/repos	root	quant-dev	开发组可读写；生产分支变更应走 Git PR。
/srv/quant/configs	root	quant-admin	仅管理员和特定服务账号可读；禁止普通成员读取。
/srv/quant/data	root	quant-research	研究数据可控读取；raw 写权限仅给数据服务。
/srv/quant/results/private	root	quant-admin	原始结果和敏感明细，仅核心服务和管理员可读。
/srv/quant/results/internal	root	quant-research	内部评估摘要、报告和可协作结果。
/srv/quant/results/public	root	quant-readonly	展示层结果、状态和脱敏摘要。
/srv/quant/logs	root	quant-ops	运维可读，普通研究人员不直接读敏感日志。

6 模块 API 文档规范

6.1 为什么必须先写接口文档

多人协作时，模块之间不能依赖口头约定或直接读取对方内部脚本。每个模块应先提供稳定的接口文档，再进行联调。接口文档是模块边界、权限边界、数据契约和任务契约的共同依据。

接口治理原则

模块可以内部快速迭代，但对外暴露的输入、输出、Job、Event 和 Manifest 必须稳定、可测试、可追踪。

6.2 INTERFACE.md 模板

每个主要模块建议提供统一模板的 INTERFACE.md。

```
# <module_name> INTERFACE

## 1. Module Responsibility
- What this module owns
- What this module does not own

## 2. Upstream Inputs
- Dataset / artifact name
- Schema
- Required fields
- Optional fields
- Validation rule

## 3. Downstream Outputs
- Output name
- Schema
- Storage location
- Visibility level

## 4. Query API
```

- Endpoint
 - Params
 - Response schema
 - Error codes
- ## 5. Job API
- Job type
 - Request schema
 - Status model
 - Output manifest
- ## 6. Emitted Events
- Event type
 - Payload schema
 - Trigger condition
- ## 7. Consumed Events
- Event type
 - Action after receiving event
- ## 8. Manifest
- Required fields
 - Artifact paths
 - allowed_consumers
- ## 9. Permission Requirement
- Service account
 - Read paths
 - Write paths
- ## 10. Smoke Test
- Minimal command
 - Expected output

6.3 模块接口文档清单

模块	接口文档重点	典型 Smoke Test
raw_data_layer	数据下载、校验、清洗、数据就绪事件、cleaned parquet manifest。	小样本数据下载与清洗。
factor_engine	因子表达式、YAML 配置、factor lake 输出、计算 Job。	单因子最小配置运行。
factor_evaluation	IC / RankIC / 分层回测评估 Job 和结果 schema。	指定 factor value + market data 评估。
factor_admission	准入规则、approve / reject 决策、factor_admitted event。	读取评估结果生成 admission summary。
factor_agent	研报解析、YAML 草稿、候选因子和评分修订循环。	输入示例文本，输出候选 YAML。
strategy_layer	single asset target position、portfolio signal、holdings schema。	输入 factor lake，输出 target_position 或 holdings。
backtest_layer	回测 Job、returns / metrics / summary / audit metadata。	跑单资产或组合最小回测。

7 API / Job API / Event 分工

7.1 三者的边界

平台模块协作采用三类机制：Query API、Job API 和 Event。三者不可混用。

类型	作用	是否触发计算	典型场景
Query API	查询已有结果、状态、摘要和 Manifest。	否	查询 factor summary、backtest metrics、job status、artifact manifest。
Job API	请求系统执行耗时任务。	是	数据清洗、因子计算、因子评估、回测、Agent 解析、模型训练。
Event	通知下游某个产物或状态已经就绪。	否	market_data_ready、factor_admitted、holdings_ready、backtest_finished。

7.2 Query API

Query API 只负责查询，不负责触发重计算。典型接口如下：

```
GET /api/jobs/{run_id}
GET /api/artifacts/{run_id}/manifest
GET /api/factors/{factor_id}/summary
GET /api/backtests/{run_id}/summary
GET /api/strategies/{strategy_id}/card
GET /api/data-quality/{dataset_id}/summary
```

Query API 的返回结果应当是授权后的投影结果，不应直接返回 private 目录路径、服务器绝对路径、完整原始日志或密钥字段。

7.3 Job API

Job API 负责接收请求、校验权限、写入审计、进入队列、分配 worker，并返回 run_id。调用方不直接执行脚本，也不登录目标服务器。

```
POST /api/jobs/data-cleaning
POST /api/jobs/factor-computation
POST /api/jobs/factor-evaluation
POST /api/jobs/factor-admission
POST /api/jobs/strategy-signal-generation
POST /api/jobs/backtest
POST /api/jobs/agent-report-parse
POST /api/jobs/build-feature-matrix
POST /api/jobs/model-training
```

7.4 Event

Event 用于模块之间的产物就绪通知。Event 不替代 Job API，也不负责查询大对象。它只传递足够让下游决定是否启动下一步的元数据。

```
market_data_ready
cleaned_data_ready
factor_values_ready
factor_admitted
strategy_signal_ready
holdings_ready
backtest_finished
model_version_ready
```

7.5 典型调用链路

数据到因子的链路

```
Data Job finished
-> Event: cleaned_data_ready
-> Factor Service receives event
-> POST /api/jobs/factor-computation
-> factor_values_ready
-> POST /api/jobs/factor-evaluation
```

Agent 到因子评估链路

```
Agent Service parses report
-> candidate factor YAML
-> POST /api/jobs/factor-evaluation
-> manifest + summary
-> Factor Card for human review
```

策略到回测链路

```
Strategy Service generates holdings
-> Event: holdings_ready
-> POST /api/jobs/backtest
-> Event: backtest_finished
-> Dashboard reads summary via Query API
```

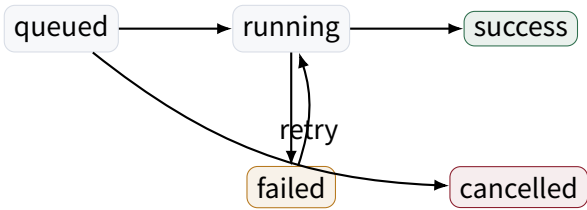
8 Job 执行、Manifest 与结果权限分层

8.1 Job Request 标准格式

所有异步任务必须使用结构化 Job Request，不允许传入任意 shell 命令。

```
{
  "job_type": "factor_evaluation",
  "requester": "svc-agent",
  "requester_group": "factor-team",
  "environment": "prod",
  "priority": "normal",
  "params": {
    "factor_id": "capex_pressure_revenue_visibility",
    "market": "US",
    "universe": "us_semiconductor",
    "start_date": "2021-01-01",
    "end_date": "2025-12-31",
    "frequency": "1d"
  },
  "visibility": "internal",
  "allowed_consumers": ["svc-agent", "svc-dashboard"],
  "reason": "evaluate candidate factor from report",
  "dry_run": false
}
```

8.2 Job 状态机



8.3 参数白名单与资源限额

任务类型	参数白名单	资源限制建议
data-cleaning	dataset、date range、source、output visibility。	生产全量清洗需管理员审批；普通任务限制日期范围。
factor-evaluation	factor_id、universe、date range、frequency、horizon。	大样本全市场评估排入 batch 队列，避免抢占实时资源。
backtest	strategy_id、holdings path、cost model、date range。	并发数限制；必须输出 summary / metrics / returns。
agent-report-parse	document id、parser profile、output visibility。	不允许返回原始研报全文给未授权用户。
model-training	feature set、model config、gpu profile、budget limit。	GPU 训练必须有预算限额和运行时长上限。

8.4 Manifest 标准

每个成功或失败的 Job 都应生成 Manifest。Manifest 是下游读取结果、审计运行版本和控制权限的唯一入口。

```
{
  "run_id": "backtest_20260502_001",
  "producer": "backtest_layer",
  "status": "success",
  "job_type": "backtest",
  "code_version": "git:abc123",
  "data_version": "workspace_data/factors/lake/20260502",
  "config_version": "sha256:...",
  "submitted_by": "svc-strategy",
  "started_at": "2026-05-02T12:00:00Z",
  "finished_at": "2026-05-02T12:04:23Z",
  "visibility": "internal",
  "artifacts": {
    "summary": "workspace_data/backtests/.../summary.json",
    "metrics": "workspace_data/backtests/.../metrics.json",
    "returns": "workspace_data/backtests/.../returns.parquet",
    "private_trace": "results/private/backtest_20260502_001/trace.parquet"
  },
  "allowed_consumers": ["strategy_layer", "dashboard_service"]
}
```

8.5 结果权限分层

层级	内容	例子	访问对象
private	完整原始结果、敏感明细、调试轨迹。	完整因子值、逐日持仓、交易明细、LLM 调用日志、原始报错栈。	管理员和授权服务账号。
internal	项目内部协作结果。	IC、RankIC、分层回测、metrics、summary、准入结论。	研究组、模块负责人、授权服务。
public	展示层投影结果。	状态、脱敏图表、简化策略卡片、是否通过审核。	Dashboard、只读角色、对外展示页面。

8.6 服务化协作方式

模块之间不直接跨服务器 SSH 执行脚本，也不直接读取对方 private 目录。推荐协作方式如下：

```
Agent Service -> Factor Evaluation Job
Data Service -> Event: cleaned_data_ready -> Factor Service
Factor Admission -> Event: factor_admitted -> Model Service
Strategy Service -> Backtest Job
Backtest Service -> Event: backtest_finished -> Dashboard
```

服务化协作必须使用 service account 调用，并且经过 RBAC、Job Request schema、参数白名单和审计日志控制。

9 迁移、上线与回滚流程

9.1 迁移目标

迁移的目标不是简单复制文件，而是将原内网服务器上的单用户代码资产，转换为腾讯云 Server A / Server B 上的可治理工程平台。

9.2 迁移流程

- 1. **原服务器盘点：**盘点 Git 状态、未提交代码、数据目录、环境依赖、定时任务、后台服务、日志、密钥和端口。
- 2. **代码 Git 化：**所有代码必须进入 Git；未提交改动建立 migration snapshot commit 和 tag。
- 3. **新服务器初始化：**创建用户组、服务账号、/srv/quant 目录、数据盘挂载和基础依赖。
- 4. **Environment 统一：**创建平台统一环境，安装根依赖和模块需要的 editable package。
- 5. **配置迁移：**将真实配置迁入 /srv/quant/configs/<module>/prod.env，不进入 Git。
- 6. **数据迁移：**raw、cleaned、workspace_data 分层迁移；优先使用 rsync dry-run 验证。
- 7. **服务部署：**使用 systemd 托管 Job API、数据任务、因子任务、回测任务和 Dashboard API。
- 8. **双跑验证：**新旧服务器并行 1-3 天，对比行数、日期覆盖、symbol 覆盖、Job 输出和日志异常。
- 9. **正式切换：**冻结旧服务器写入，最后一次同步，切换 API / Job 入口。
- 10. **回滚窗口：**旧服务器至少保留两周，关键节点创建腾讯云快照。

9.3 迁移验收 Smoke Test

```
# 多因子组合全链路 demo
python demo/all_pipeline_demo/run_all_pipeline_demo.py
pytest demo/all_pipeline_demo/test_all_pipeline_demo.py -q

# 单资产择时全链路 demo
```

```
python demo/all_pipeline_single_asset_demo/run_single_asset_pipeline_demo.py
pytest demo/all_pipeline_single_asset_demo/test_single_asset_pipeline_demo.py -q

# 回测层测试
pytest backtest_layer/tests -q -m "not integration"
```

10 监控、告警与后续维护人员要求

10.1 systemd 服务托管

生产常驻服务不得长期使用 `nohup python xxx.py &` 管理。统一使用 `systemd`，便于开机自启、日志查询、失败重启和权限控制。

```
[Unit]
Description=Quant Job API Service
After=network.target

[Service]
User=svc-jobapi
Group=quant-ops
WorkingDirectory=/srv/quant/repos/quant society_backend
EnvironmentFile=/srv/quant/configs/job_api/prod.env
ExecStart=/srv/quant/envs/platform/bin/python -m job_api.main
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

常用命令：

```
sudo systemctl status quant-job-api
sudo systemctl restart quant-job-api
journalctl -u quant-job-api -f
```

10.2 监控与告警清单

监控对象	指标	告警条件
服务器资源	CPU、内存、磁盘、网络、inode。	CPU 长期大于 90%，磁盘大于 80%，内存持续高水位。
数据链路	数据新增时间、行数、symbol 覆盖、文件完整性。	数据长时间无新增、日期缺口、parquet 校验失败。
Job 系统	queued / running / failed 数量、耗时、重试次数。	Job 失败率升高、队列堆积、单任务超时。
API 服务	p95 延迟、5xx 错误率、请求量。	API p95 超过阈值，5xx 连续升高。
对象存储	上传成功率、容量、费用、归档延迟。	上传失败、容量异常增长、归档任务失败。
安全审计	SSH 登录、sudo 操作、配置修改。	异常 IP 登录、非授权 sudo、prod.env 改动。

10.3 后续维护人员分级

维护层级	权限范围	典型任务
L1 只读维护	查看 public / internal summary、读取普通日志。	查看 Job 状态、执行 smoke test、汇报异常。
L2 模块维护	修改开发分支、提交 PR、运行 dev job。	修复模块 bug、补接口文档、跑开发环境任务。
L3 服务维护	重启授权服务、排查 systemd、处理普通失败任务。	重启非核心服务、查看 journalctl、处理 Job retry。
L4 管理维护	修改 prod env、管理密钥、执行迁移和回滚。	生产配置变更、权限回收、快照恢复、正式切换。

10.4 后续维护人员必备技术栈

- Linux 用户、权限、进程与日志；
 - SSH 公钥登录与安全组；
 - Git 分支、PR、tag 与回滚；
 - Python Environment 与依赖锁定；
 - .env.example 与 prod.env；
- systemd 与 journalctl；
 - Parquet、DuckDB、workspace_data；
 - API / Job API / Event / Manifest；
 - private / internal / public 结果权限；
 - 云服务器、对象存储、快照与备份。

附录：推荐落地顺序

阶段	目标	交付物
Week 1	完成服务器初始化、用户组、服务账号、目录权限、统一 Environment。	Server A / B 可登录、可运行 smoke demo、配置目录和数据目录就绪。
Week 2	完成模块接口文档初版。	每个主要模块提供 INTERFACE.md ，明确输入、输出、Job、Event 和 Manifest。
Week 3	完成 Job API MVP。	支持 factor-evaluation、backtest、data-cleaning 等核心 Job，返回 run_id 和 manifest。
Week 4	完成迁移双跑和监报告警。	新旧服务器结果对齐，systemd、日志、Job 监控、回滚方案就绪。

最终判断

该平台的服务器治理重点不是单纯购买和管理两台机器，而是建立模块之间的标准协作方式：Query API 用于查询，Job API 用于执行，Event 用于通知，Manifest 用于交付结果，权限分层用于控制可见性。只有先完成这套工程契约，后续多人协作、Agent 自动化、因子评估、策略回测和服务化展示才不会变成不可维护的脚本堆叠。